

Your Agent Escaped Without Escaping the Sandbox

The harness stays in the dev sandbox,
poisons what the CI runner trusts,
and the runner promotes that bug into the prod sandbox.

Yossi Eliaz, PhD • Incredibuild • 21 October 2026

Spoken notes generated from slides/talk.tex. Slides 1–12 are the talk. Slides 13–18 are for questions.

1. What changed when the test went green?

00:00-01:00

Show FAIL, then PASS. Ask what changed.

What changed when the test went green?

The recording starts at FAIL and ends at PASS. Same setup, one worker step in between. I am Yossi Eliaz. For the next fourteen minutes we stay on that pair of results and name the file that moved. By the end, the room should be able to point at that file and say why PASS appeared.

The worker in the recording is a script. The program is a small command-line model of an access check. It prints an integer. It does not serve an HTTP admin page, and this talk does not show a GitHub merge. The signal on the slide is the checker's own FAIL and PASS.

2. No login prints 200

01:00-02:30

Name the case. Then the two numbers.

One case is enough to see the failure. admin:none is the model's name for the admin route with no login. The required status is 401. The program prints 200 for every input, including this one.

A correct program prints 401 for no login, 200 for a real admin, 403 for an ordinary user, 200 for the public route, and 401 for an expired login. This program prints 200 five times. The checker will compare those integers with a file. Until that file is in view, FAIL and PASS are just two words on a slide.

3. The checker is protected

02:30-04:00

Denied first. Then the line it reads.

The worker tries to replace the checker with a program that prints PASS. The read-only mount returns DENIED. The checker hash is f5d8d3468bbe before that step and the same value after it.

The checker then loads expected.json from its working directory. That file is in the workspace. The protection on the executable does not cover the file the executable uses to decide success. With the

honest file, this same checker prints FAIL and exits 1. The executable protection worked. The next slide is the input that protection leaves writable.

4. Only the answer file moved

04:00-06:00

Read across the row. Stop on PASS.

Read the row. Program 76fcb0904f51 both times. Checker f5d8d3468bbe both times. The answer file moves from 0494f1a01dee to 390e5e764916. The verdict moves from FAIL to PASS.

The diff is the required status for admin:none, 401 becoming 200, and the same change on the other four rows. The worker changed what counted as correct. The checker executed, compared the program with that file, and the numbers matched.

The worker changed what counted as correct. The checker compared the program with the new file, and the numbers matched.

5. What the three hashes say

06:00-07:00

One sentence per hash.

The unchanged program hash means this comparison did not repair the program. The unchanged checker hash means this comparison did not edit that executable. The changed answer-file hash names the dependency that moved.

Hashing the altered answers does not make them authoritative. Their identity and their owner have to come from the trusted policy. In this run the owner of the new file was the worker. If you only hash the file after the worker writes it, you have a name for the new bytes and you still have the worker as the author of the criteria.

6. The file defines success

07:00-08:00

Point at the load. Then at who can write it.

This is why the green result happened. expected.json looks like ordinary project data. The checker gives that file the authority to define success. The significance comes from the decision that reads it.

The worker can write the workspace. The checker runs with that workspace as its working directory. So a workspace write is a write to the acceptance criteria. An inventory of protected executables still leaves this input on the worker's side of the line.

The worker here is a script with that write in it. The recording shows the action is permitted. It does not measure whether a particular model would choose it. For an engineer or a CISO, the useful question is which input the trusted decision treats as the definition of success, and who is allowed to write that input. A list of locked binaries still leaves that question open on the table tonight.

7. Controller-owned criteria

08:00-09:15

REJECT the same program. The fix is the next slide.

The repair uses a different acceptance path. The controller runs the frozen program and compares the integers with cases it owns. It does not load `expected.json` from the workspace.

Program `76fcb0904f51`, the one that prints 200, is REJECT. The demonstrated edit no longer moves this verdict. This path is the controller's comparison. It is a separate implementation from the workspace checker whose hash is `f5d8d3468bbe`.

8. The corrected program

09:15-10:30

Different hash. Five numbers. ACCEPT.

The corrected program has hash `5ead8eab70e0`. It returns 401 for no login, 200 for a real admin, 403 for an ordinary user, 200 for the public route, and 401 for an expired login. The controller accepts it.

The repair stops the demonstrated edit and keeps the legitimate task. The accepted program is not the program from the failing run. Its hash is different, and the criteria are the controller's cases.

9. One unused approval

10:30-12:00

Say unused. Then denied, published, denied.

Acceptance and publication are separate. The approval on this slide was issued for the corrected program and has not been used yet. The first attempt offers the bad program under that fresh approval. The gate returns DENIED. The second attempt offers the approved bytes and returns PUBLISHED. The third attempt offers those bytes again and returns DENIED.

Because the first denial happens while the approval is unused, the refusal is the byte check. A used approval would also deny a replay, and that would hide a bad swap. An earlier version of this gate consumed the approval first. The published row is the corrected program, compared by the controller and stored as those bytes. The first row is the important one for the question people will ask: the denial happens before the approval is spent, so replay protection is not what rejected the bad program.

10. Three questions for another system

12:00-13:15

Read the three questions. Do not name a vendor.

Take these three questions to another system. Who can change the inputs that define acceptance? What evidence identifies the candidate, the checker, and the criteria used for this decision? Does publication consume exactly the approved bytes?

These questions inspect a pipeline. They do not by themselves show that another product has this bug. On our recording, the worker can change `expected.json`, and that file defines acceptance for the workspace checker. The hashes name the candidate, the checker, and the criteria for this comparison. The unused approval shows that publication stored the approved bytes, because the bad bytes were refused while that approval was still unused.

11. Next experiment

13:15-13:40

Read it as a question. Label it unfinished.

Leave this up long enough to read. Once the checker and the answer file are both protected, which remaining input can the worker use to change the verdict? That experiment is not in this recording. If a later run finds such an input, that input becomes the result. If none does, the repair held for the inputs we tried. I am leaving it labeled as the next experiment so the boundary of this talk stays visible.

12. The worker changed what counted as correct

13:40-14:00

PASS beside the unchanged program. Stop.

The worker changed what counted as correct. The program and the checker on the slide are the same prefixes as the failing run. The answer file is the one that moved, and the checker printed PASS. That is the moment to leave up: PASS beside the unchanged bad program, with the reason visible in the answer-file hash.

The repository has the source and the Linux recordings. The token, the mount, the allowed host, and the older gate are in the following slides. Each has its own prerequisite. The older gate flaw needs access to a controller-side object. The worker path to that flaw is not in this recording. Thank you.

Questions

13. The five policy cases

Questions

Use only in questions.

The handler takes a route and an authentication label and prints an integer. The runner checks these five. The repaired handler matches all five. This is the model of the check that promotes into the prod sandbox.

14. Runner token and the dev disk

Questions

Use only in questions.

The token case is loopback HTTP. The broad fixture stands in for the runner secret. The task fixture stands in for a job-scoped token. The mount case watches the host file after an ordinary write from the dev sandbox. This job's output stays writable.

15. Allowed host, wrong account

Questions

Use only in questions.

The fixture service permits both account paths, the way one allowed deploy host holds more than one operation. The delivery record shows the bytes arrived. The repair moves the POST onto the runner and removes the dev sandbox's direct route.

16. An earlier promote stored a marker

Questions

Use only in questions.

The archived case mutates a controller-side object, keeps a stale digest, and the old gate records a publication marker. The current promote derives the digest from immutable bytes and stores those bytes. Reaching the old flaw takes access to the controller object.

17. What was recorded

Questions

Use only in questions.

The records bind their source. The dev sandbox recordings use bubblewrap on a Boat Ubuntu machine. The harness under test is a script. The prod step in the recording is the runner's publication of exact bytes, the shape of what a prod sandbox is allowed to execute. The forked-disk slide follows from that workspace file. The timings are a delivery budget.

18. Reproduce the factory

Questions

Use only in questions.

Run the fixtures on disposable Linux with bubblewrap. The deck reads those recordings. Poisoned pipeline execution is the public name for a CI job that runs an attacker-influenced file. Anthropic's reward-hacking study is prior motivation and its measurements stay with that study. Reproducing the talk needs no GitHub token.